Programowanie obliczeń komputerowych

Dodatek: Integracja kodu Python – C# w środowisko języków dynamicznych platformy .NET

Funkcjonalność podsystemu DLR

- Umożliwia programowanie dynamiczne charakterystyczne dla języków skryptowych, takich jak Python, Ruby
- Pozwala na dowolne dynamiczne działania (np. arytmetyczne) na obiektach o typie nieokreślonym, a nawet pozwala na dynamiczną modyfikację struktury tych typów
- Te i inne operacje dynamiczne są wykonywane w miejscu ich użycia jedynie na samej nazwie zmiennej bez znajomości typu obiektu
- Definicja wartości zmiennej, a nawet deklaracja jej typu, mogą być w tym miejscu zupełnie niedostępne dla kompilatora, gdy obiekty wartości są tworzone w innym podzespole aplikacji
- Definicje mogą być nawet napisane w innym języku, np. użycie jest w programie C#, a definicja znajduje się w skrypcie Python
- Wszystkie takie operacje dynamiczne muszą być wyłącznie zrealizowane w czasach działania aplikacji przez podsystem DLR

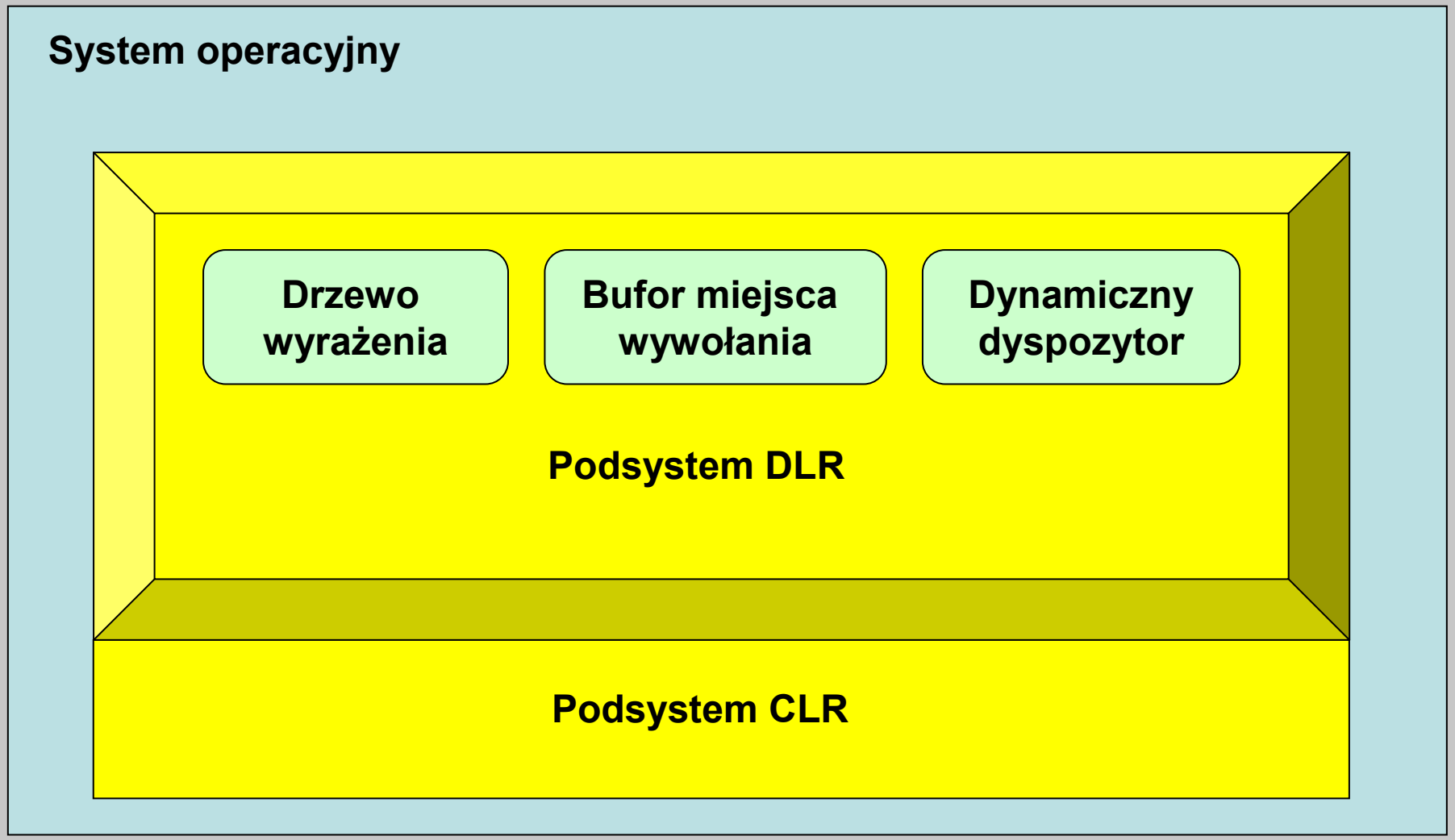
Funkcjonalność podsystemu DLR

- Pełne wsparcie dla takich operacji oferuje jedynie język Visual C# wraz ze środowiskiem języków dynamicznych DLR
- Począwszy od wersji 4.0 języka w Visual Studio 2010 podsystem ten stanowi istotne rozszerzenie wspólnego środowiska uruchomieniowego CLR dla języków platformy .NET
- W tym celu do gramatyki języka C# wprowadzono osobny typ dynamiczny specyfikowany słowem kluczowym **dynamic**
- Ponadto do biblioteki klas platformy .NET dodano klasy wsparcia dla programowania dynamicznego
- Podsystem DLR stanowiący część platformy .NET pozwala przy tym na implementację języków dynamicznych bez potrzeby tworzenia osobnych analizatorów językowych, generatorów kodów i innych klasycznych narzędzi towarzyszących implementacji języka
- Do użycia DLR w danym języku niezbędna jest jedynie możliwość dynamicznego generowania drzewa składniowego wyrażeń

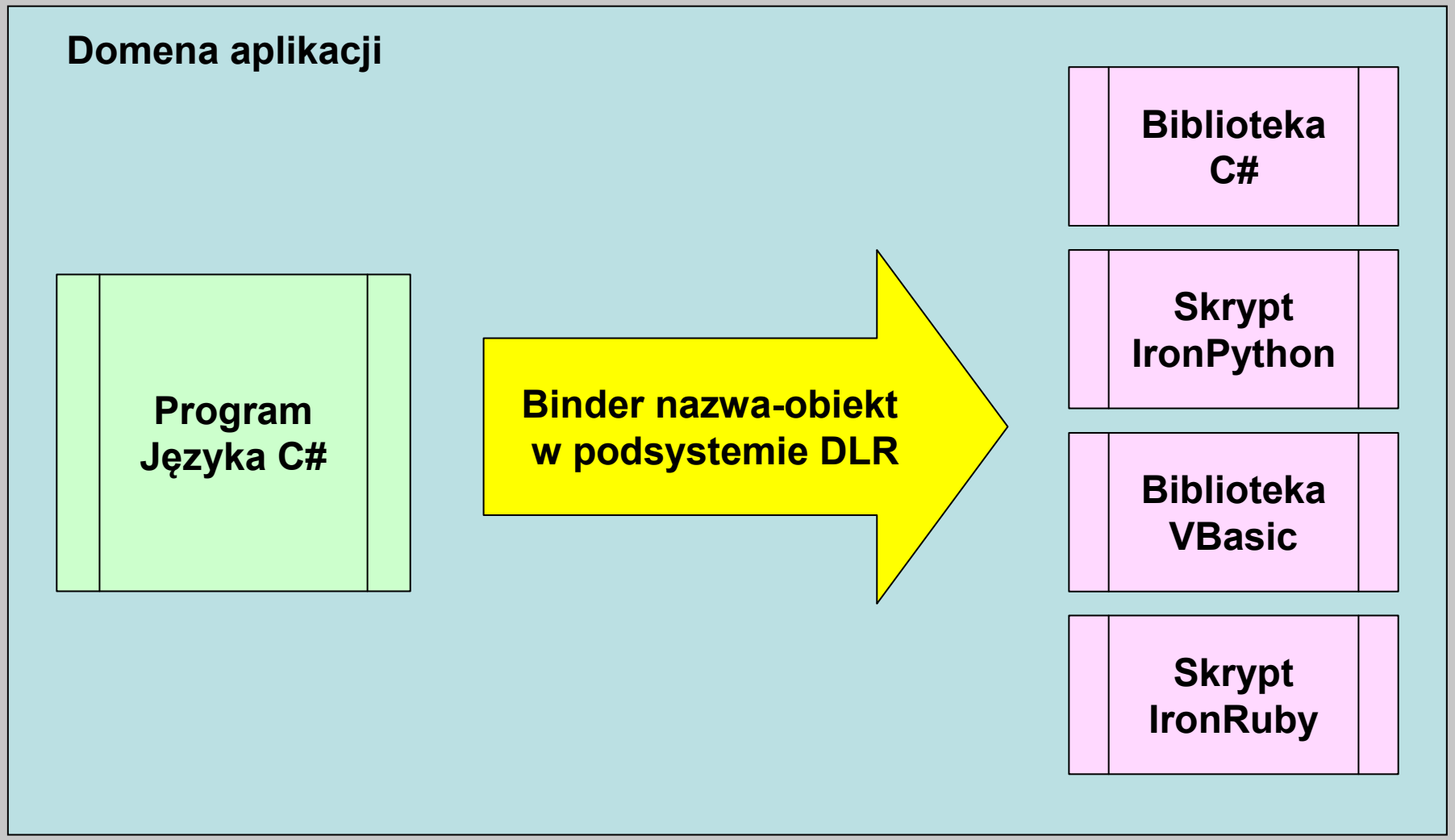
Funkcjonalność podsystemu DLR

- Dzięki temu można reprezentować w czasie działania aplikacji fragment kodu programu zawierający dowolne operacje dynamiczne
- Drzewo to razem ze dynamicznym dyspozytorem i buforowaniem miejsca wywołania dynamicznej operacji pozwala na dynamiczne wiązanie nazw różnych elementów programu z ich definicjami
- Zachodzi to nawet bez konieczności podawania deklaracji nazwy zmiennej z typem wartości, a widocznej w miejscu użycia nazwy
- Obiektami o takim dynamicznym wiązaniu nazw mogą być typy, dane, procedury, klasy i ich instancje
- Co więcej, użycie i definicje tych obiektów mogą należeć do rożnych języków używanych w ramach tej samej aplikacji
- Dzięki temu podejściu platforma .NET usprawnia zadania odpowiedzialne za analizę i generowanie kodu języka dynamicznego, aby implementacja tego języka skupiała się jedynie na jego wyjątkowych własnościach językowych

Architektura podsystemu DLR



Dynamiczna integracja języków DLR



Prosty przykład w języku C#

```
static int Main()
{
 var a = 3.14;
 var b = 3.14;
 a = 123;
 a = a+1;
 // a = "tekst";

 dynamic x = 3.14;
 dynamic y = 3.14;
 x = 123;
 x = x+1;
 x = "tekst";

 dynamic obiekt = new ExpandoObject();
 obiekt.pole = 234;
 obiekt.metoda = (Func<int, int>)delegate (int x) {
  return 2 * x + 1;
 };
 dynamic wynik = obiekt.pole+obiekt.metoda(12, 15);

 return 0;
}
```

różne obiekty o tej samej wartości i statycznie rozpoznanym typie double

brak możliwości swobodnego dowiązywania nazwy do innego obiektu

ten sam buforowany obiekt o dynamicznie określonym typie double

pełna swoboda w dowiązywaniu nazwy do innych obiektów

utworzenie pustego obiektu o dynamicznie wstrzykiwanych polach oraz metodach poprzez delegacje lub lambdy

Przykład programu w języku C#

```
static int Main()
{
 try
 {
  dynamic obiekt = UtworzNieznanyObiekt();
  obiekt.pole = 123;
  dynamic wynik = obiekt.metoda(12, 15);
  Console.WriteLine("{0}-metoda = {1}", wynik.GetType(), wynik.ToString());
  dynamic wartosc = obiekt.pole;
  Console.WriteLine("{0}-pole = {1}", wartosc.GetType(), wartosc.ToString());

  dynamic a = UtworzNieznanyObiekt1();
  dynamic b = UtworzNieznanyObiekt2();
  dynamic c = a+b;
  Console.WriteLine("{0}-dodawanie = {1}", c.GetType(), c.ToString());
 }
 catch
 {
  Console.WriteLine("Blad w programowaniu dynamicznym!");
 }
 return 0;
}
```

modyfikacja typu obiektu o nieznanej klasie, zdefiniowanej gdzieś w domenie

działania na obiektach nieznanych typów

Przykład biblioteki w języku C#

```
class DynamicznaKlasa : DynamicObject {
 Dictionary<string, object> skladniki = new Dictionary<string, object>();
 public override bool TryGetMember(GetMemberBinder binder, out object value)
 { return skladniki.TryGetValue(binder.Name, out value); }
 public override bool TrySetMember(SetMemberBinder binder, object value)
 { skladniki[binder.Name] = value; return true; }
 public override bool TryInvokeMember(InvokeMemberBinder binder, object[] args,
                                      out object result)
 {
  if (binder.Name == "metoda") {
   int x = (int)args[0];
   int y = (int)args[1];
   result = 1.2*(x + y);
   return true;
  } else {
   result = null;
   return false;
  }
 }
}

static dynamic UtworzNieznanyObiekt()
{ return new DynamicznaKlasa(); }
```

```
static int opcja;

static dynamic UtworzNieznanyObiekt1()
{
 if (opcja==1) return 1234;
 else if (opcja==2) return 0.567;
 else return "abc";
}

static dynamic UtworzNieznanyObiekt2()
{
 if (opcja==1) return 3.14;
 else return "blad";
}
```

Przykład integracji C# - Python

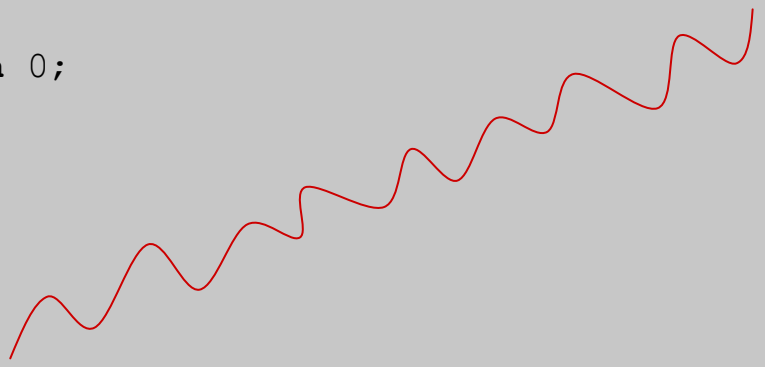
```
static int Main() // Program C#
{
 var parser = Python.CreateEngine(); // utworzenie parsera jezyka Python
 var zakres = parser.CreateScope(); // utworzenie pustego zakresu deklracyjnego
 var zmienna = 123; // zdefiniowanie obiektu w programie C#
 zakres.SetVariable("_zmienna", zmienna); // deklaracja do zakresu skryptu Python

 var skrypt = Console.ReadLine(); // odczytanie sciezki do pliku z skryptem
 var modul = parser.CreateScriptSourceFromFile(skrypt); // utworzenie modulu Py

 dynamic rezultat = modul.Execute(zakres); // wykonanie modulu przez interpreter
 Console.WriteLine("rezultat: {0}", parser.Operations.Format(rezultat));

 dynamic wartosc = zakres.GetVariable("wartosc_"); // uzycie z zakresu skryptu
 Console.WriteLine("{0} wartosc = {1}", wartosc.GetType(), wartosc.ToString());

 return 0;
}
```



```
# Skrypt Python

print("zmienna =",_zmienna)
_zmienna = 3.14

wartosc_ = 567
11+12*13
```